

Ch03. word2vec



■ 주요 내용

- 추론 기반 기법과 신경망
- 단순한 word2vec
- 학습 데이터 준비
- CBOW 모델 구현
- word2vec 보충

word2vec

■ 추론 기반 기법

- 단어의 분산 표현
- 신경망을 이용하여 추론을 하는 기법

■ 이번 장에서 다룰 내용

- '단순한' word2vec 구현하기
 - 처리 효율을 희생하는 대신 이해하기 쉽게 구성

3.1 추론 기반 기법과 신경망

- 단어를 벡터로 표현하는 방법
 - 분포 가설이 배경임
 - 통계 기반 기법
 - 추론 기반 기법

- 이번 절 수행
 - 통계 기반 기법의 문제점
 - 추론 기반 기법의 이점을 거시적 관점에서 설명
 - word2vec의 전처리를 위해 신경망으로 단어 처리 예 수행

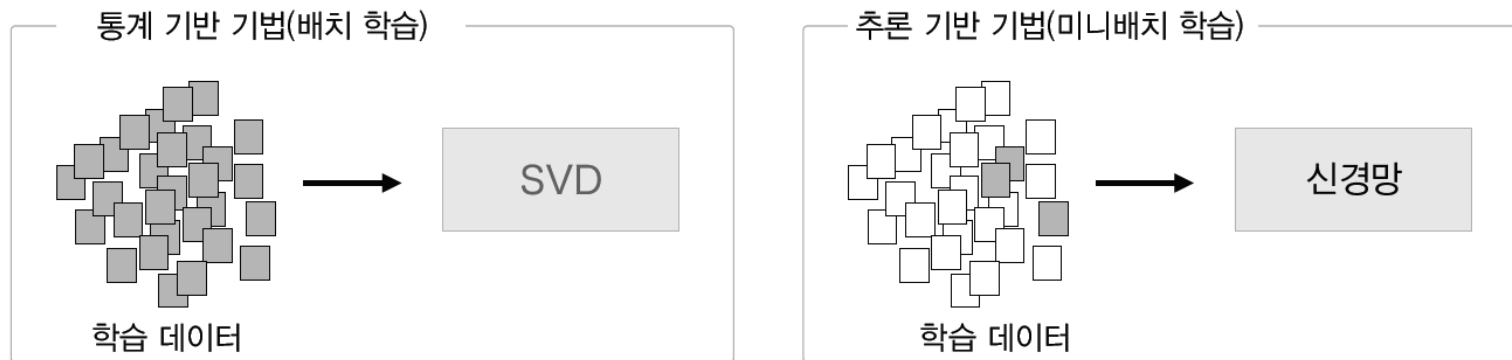
3.1.1 통계 기반 기법의 문제점

- 통계 기반 기법의 문제점
 - 주변 단어의 빈도로 단어 표현 -> 동시발생 행렬 -> 밀집 벡터 (SVD 적용)
 - 대규모 말뭉치를 다룰 때 문제가 발생함
 - 영어의 어휘 수 100만개
 - 통계 기반 기법에서는 '100만x100만' 행렬
 - 이 거대 행렬에 SVD를 적용하는 일은 현실적이지 않음.
 - 말뭉치 전체의 통계를 이용해 단 1회의 처리만에 단어의 분산 표현 얻음

3.1.1 통계 기반 기법의 문제점 (1)

- 통계 기반 기법과 추론 기반 기법 비교
 - 추론 기반 기법은 미니배치로 학습하는 것이 일반적임
 - 미니배치 학습은 신경망이 한번에 소량의 학습 샘플씩 반복해서 학습하여 가중치를 갱신함
 - 통계 기반 기법은 학습 데이터를 한꺼번에 처리함(배치 학습)
 - 추론 기반 기법은 학습 데이터의 일부를 사용하여 순차적으로 학습
 - 계산량이 큰 작업을 학습 가능, 여러 GPU를 이용한 병렬 계산도 가능

그림 3-1 통계 기반 기법과 추론 기반 기법 비교



3.1.2 추론 기반 기법 개요

■ 추론이란

- 주변 단어가 주어졌을 때 "?"에 들어갈 단어를 추측하는 작업

그림 3-2 주변 단어들을 맥락으로 사용해 "?"에 들어갈 단어를 추측한다.

you ? goodbye and I say hello.

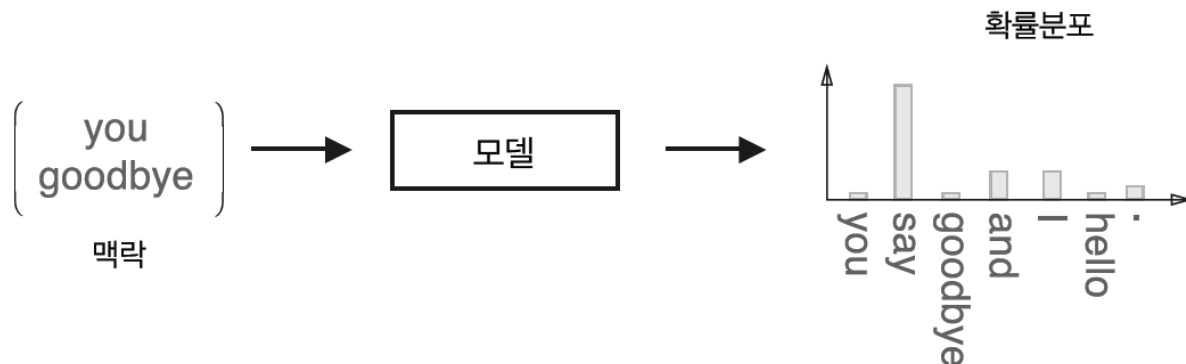


- 추론 문제를 반복해서 풀면서 단어의 출현 패턴을 학습함

■ 추론 기반 기법

- 모델로 신경망을 사용함

그림 3-3 추론 기반 기법: 맥락을 입력하면 모델은 각 단어의 출현 확률을 출력한다.



3.1.3 신경망에서의 단어 처리

- 신경망을 이용한 단어 처리
 - 단어를 고정길이 벡터로 변환
 - 원핫(one-hot) 표현 (원핫 벡터) – 벡터의 원소중 하나만 1이고 나머지는 0인 벡터
- 원핫 표현
 - 한 문장 짜리 말뭉치 “You say goodbye and I say hello.”
 - 단어는 텍스트, 단어 ID 그리고 원핫 표현 형태로 나타냄

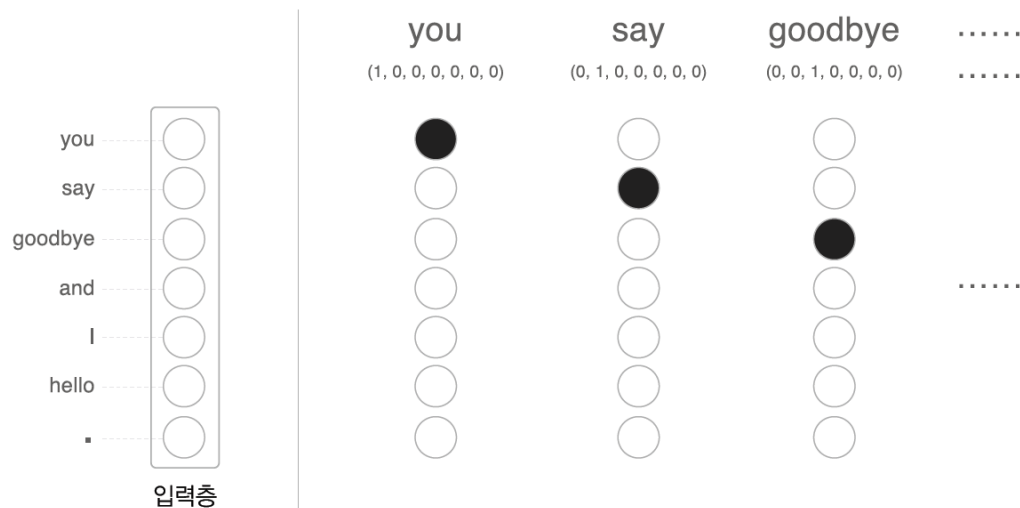
그림 3-4 단어, 단어 ID, 원핫 표현

단어(텍스트)	단어 ID	원핫 표현
$\begin{pmatrix} \text{you} \\ \text{goodbye} \end{pmatrix}$	$\begin{pmatrix} 0 \\ 2 \end{pmatrix}$	$\begin{pmatrix} (1, 0, 0, 0, 0, 0, 0) \\ (0, 0, 1, 0, 0, 0, 0) \end{pmatrix}$

3.1.3 신경망에서의 단어 처리 (1)

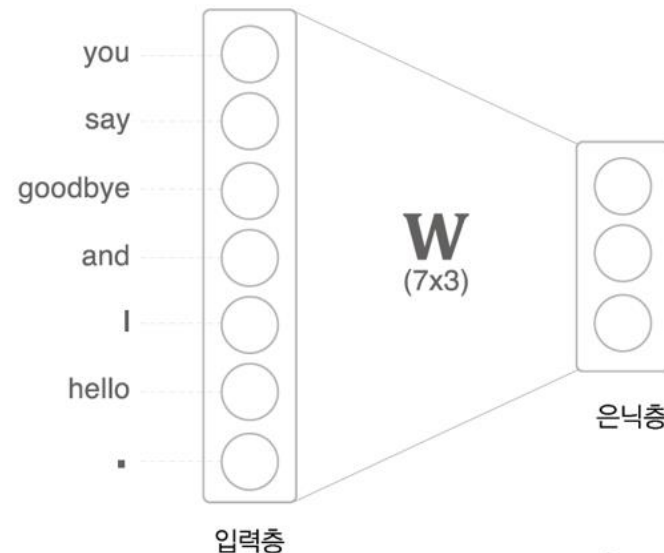
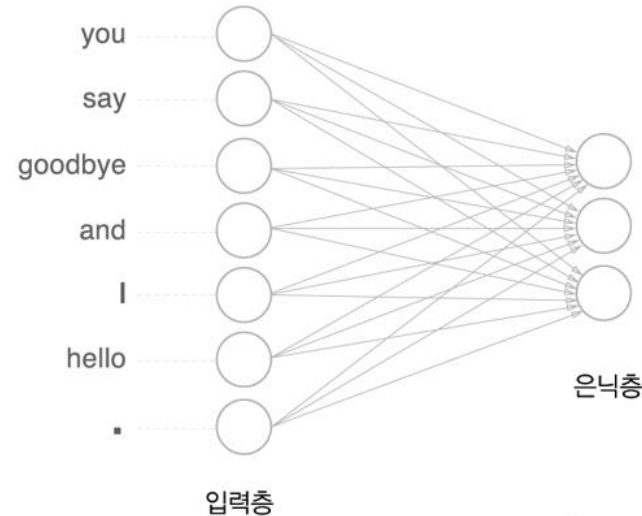
- 단어를 원핫 표현으로 변환하는 방법
 - 총 어휘 수만큼의 원소를 갖는 벡터를 준비
 - 단어를 고정 길이 벡터로 변환
 - 인덱스가 단어 ID와 같은 원소를 1, 나머지는 모두 0으로 설정
 - 입력층 뉴런은 총 7개가 차례로 7개의 단어들에 대응
 - 단어를 벡터로 나타내고, 신경망을 구성하는 계층들은 벡터를 처리함

그림 3-5 입력층의 뉴런: 각 뉴런이 각 단어에 대응(해당 뉴런이 1이면 검은색, 0이면 흰색)



3.1.3 신경망에서의 단어 처리 (2)

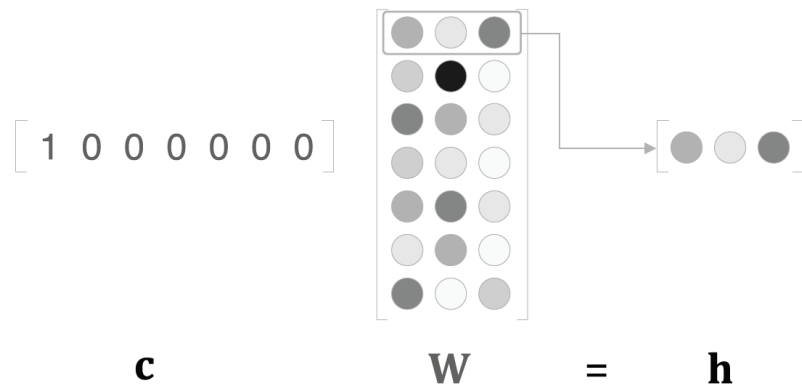
- 완전연결계층에 의한 변환
 - 입력층의 각 뉴런은 7개의 단어 각각에 대응
- 완전연결계층에 의한 변환을 단순화한 그림
 - 완전연결계층의 가중치를 7×3 크기의 W 행렬로 표현



3.1.3 신경망에서의 단어 처리 (3)

- 완전연결계층의 의한 변환 코드 작성
 - 단어 ID가 0인 단어를 원핫 표현으로 표현한 다음 완전연결계층을 통과시켜 변환함
 - 완전연결계층의 계산은 행렬 곱으로 수행
 - c 는 원핫 표현이고, c 와 W 의 행렬 곱은 가중치의 행벡터 하나를 뽑아낸 것임.

그림 3-8 맥락 c 와 가중치 W 의 곱으로 해당 위치의 행벡터가 추출된다(각 요소의 가중치 크기는 흑백의 진하기로 표현).



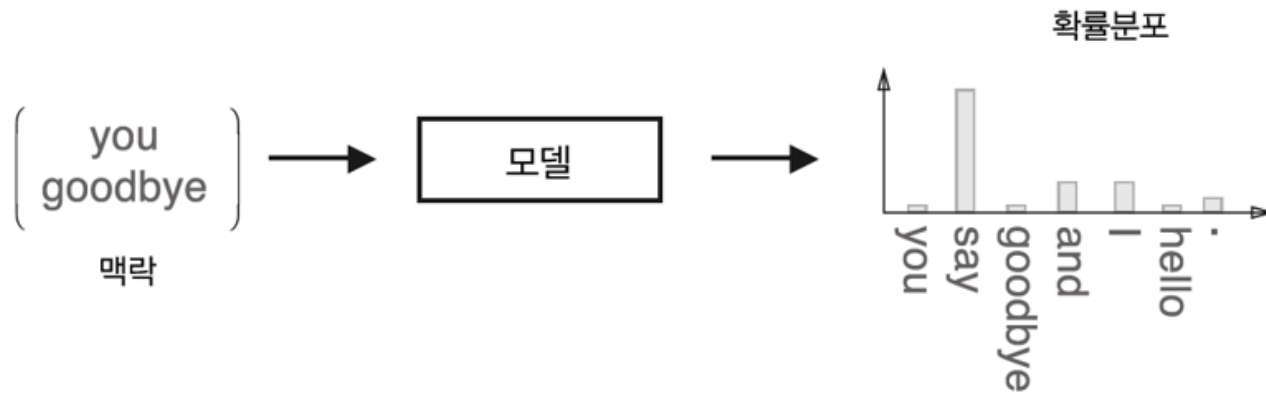
```
import numpy as np

c = np.array([[1, 0, 0, 0, 0, 0, 0]])
W = np.random.randn(7,3)
h = np.matmul(c, W)
print(h)
```

$[[\ 0.0416468 \ -0.37015816 \ -0.04268087]]$

3.2 단순한 word2vec

- 모델을 신경망으로 구축



- CBOW(Continuous Bag-Of-Words) 모델
 - word2vec 에서 사용되는 신경망

3.2.1 CBOW 모델의 추론 처리

■ CBOW 모델

- 맥락으로부터 타깃을 추측하는 용도의 신경망 (맥락은 "you" 같은 단어들의 목록)
- 가능한 정확하게 추론하도록 훈련시켜서 단어의 분산 표현을 얻어냄

■ CBOW 모델의 입력은 맥락임

■ CBOW 모델의 신경망 구조

- 입력층이 2개가 은닉층을 거쳐 출력층에 도달

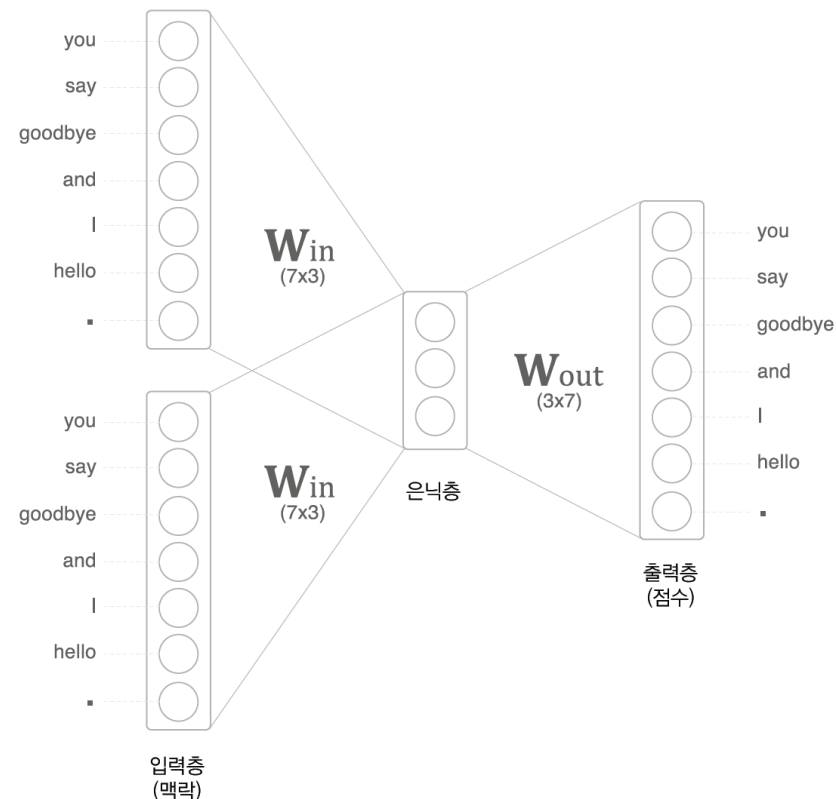
■ 은닉층 뉴런

- 입력층의 완전연결계층에 의해 변환값이 됨
- 입력층이 여러 개이면 전체를 평균함

■ 출력층의 뉴런은 총 7개임

- 뉴런 하나하나가 각각의 단어에 대응함
- 단어의 점수가 높을 수록 출현 확률도 높아짐
- 점수는 확률로 해석되기 전의 값
- 점수에 소프트맥스 함수를 적용해 확률 얻음

그림 3-9 CBOW 모델의 신경망 구조



3.2.1 CBOW 모델의 추론 처리(1)

■ 단어의 분산 표현

- 입력층에서 은닉층으로의 변환은 완전연결계층(가중치는 w_{in})에 의해서 이뤄짐
- 완전연결계층의 가중치 w_{in} 은 7x3 행렬

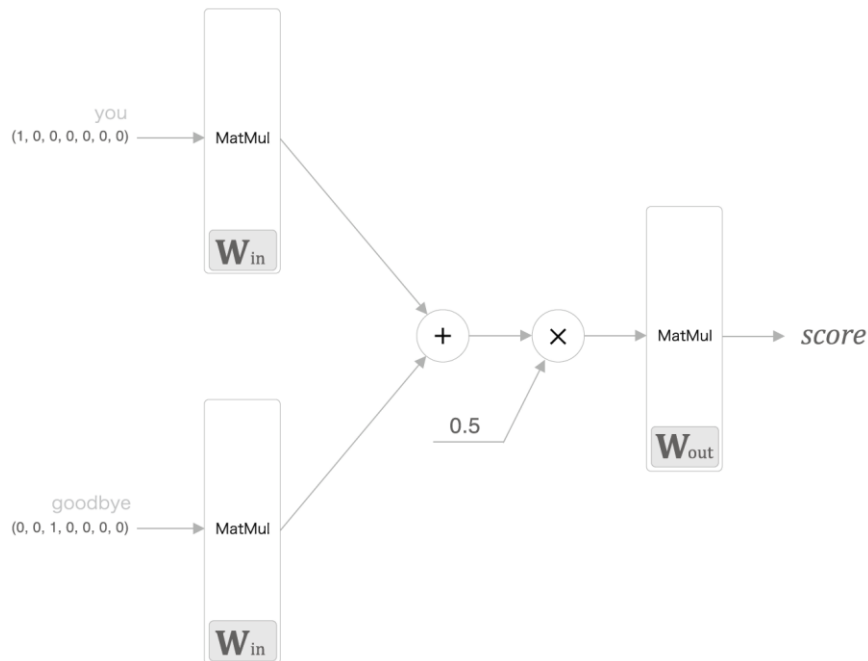


- 가중치 w_{in} 의 각 행에는 해당 단어의 분산 표현이 담겨 있음
- 학습을 진행할수록 맥락에서 출현하는 단어를 잘 추측하는 방향으로 이 분산 표현들이 갱신됨
- 이렇게 해서 얻은 벡터는 단어의 의미도 잘 녹아들어 있음

3.2.1 CBOW 모델의 추론 처리(2)

- 계층 관점에서 본 신경망
 - 2개의 MatMul 계층이 있고, 두 계층의 출력이 더해짐
 - 더해진 값에 0.5를 곱하면 평균이 되며, 이 평균이 은닉층 뉴런이 됨
 - 은닉층 뉴런에 또 다른 MatMul 계층이 적용되어 점수(score)가 출력

그림 3-11 계층 관점에서 본 CBOW 모델의 신경망 구성: MatMul 계층에서 사용하는 가중치(W_{in} , W_{out})는 해당 계층 안으로 넣었음



3.2.1 CBOW 모델의 추론 처리(3)

- CBOW 모델을 파이썬으로 구현
 - 필요한 가중치들 (W_{in} 과 W_{out})을 초기화
 - 입력층 처리 MatMul 계층을 맥락 수만큼 생성
 - 출력층 측의 MatMul 계층은 1개만 생성
 - 입력층 측의 MatMul 계층은 W_{in} 공유
 - 입력층 측의 MatMul 계층들의 forward() 호출하여 중간 데이터 계산
 - 출력층 측의 MatMul 계층을 통과시켜 각 단어의 점수를 구함
- CBOW 모델은 활성화 함수를 사용하지 않음

```
# coding: utf-8
import sys
sys.path.append('..')
import numpy as np
from common.layers import MatMul

# 샘플 맥락 데이터
c0 = np.array([[1, 0, 0, 0, 0, 0, 0]])
c1 = np.array([[0, 0, 1, 0, 0, 0, 0]])

# 가중치 초기화
W_in = np.random.randn(7, 3)
W_out = np.random.randn(3, 7)

# 계층 생성
in_layer0 = MatMul(W_in)
in_layer1 = MatMul(W_in)
out_layer = MatMul(W_out)

# 순전파
h0 = in_layer0.forward(c0)
h1 = in_layer1.forward(c1)
h = 0.5 * (h0 + h1)
s = out_layer.forward(h)
print(s)
```

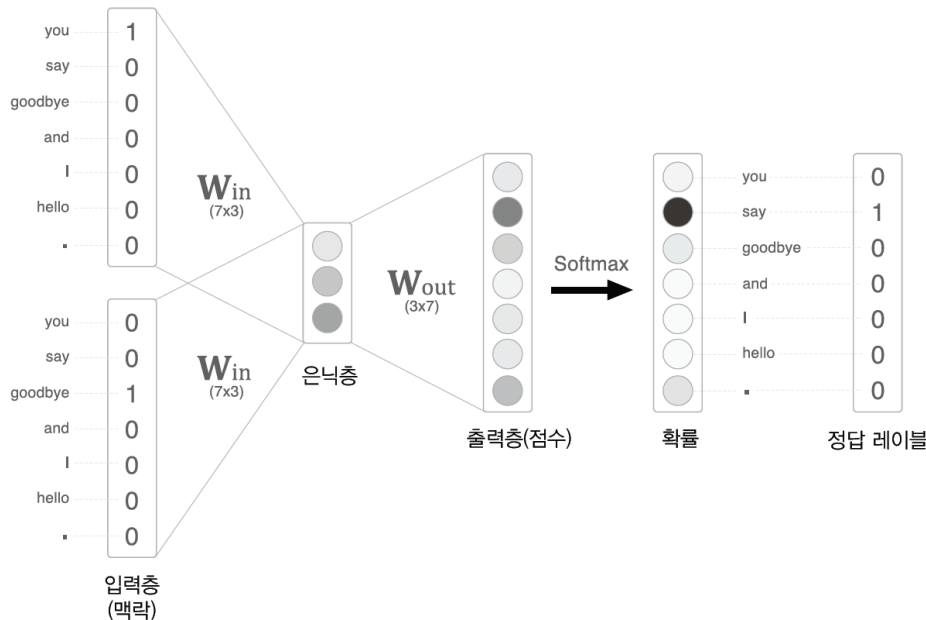
```
[[ 0.06975809 -0.91013665 -1.0766591  1.21159497  0.15774698  0.59340646
 -0.23086033]]
```


3.2.2 CBOW 모델의 학습

■ CBOW 모델의 학습

- 출력층에서 각 단어의 점수에 소프트맥스 함수를 적용하면 확률을 얻음
- 확률은 맥락이 주어졌을 때 그 중앙에 어떤 단어가 출현하는지를 나타냄
- CBOW 모델의 학습에서는 올바른 예측을 할수 있도록 가중치를 조정함
- 가중치 w_{in} 에 단어의 출현 패턴을 파악한 벡터가 학습됨

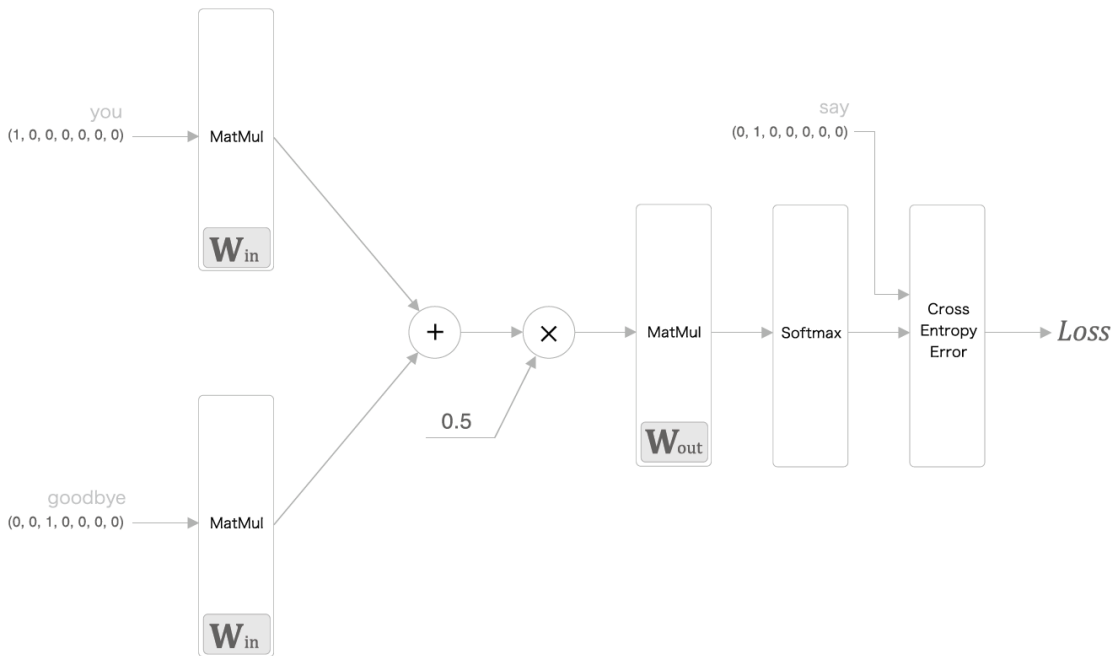
그림 3-12 CBOW 모델의 구체적인 예(노드 값의 크기를 흑백의 진하기로 나타냄)



3.2.2 CBOW 모델의 학습(1)

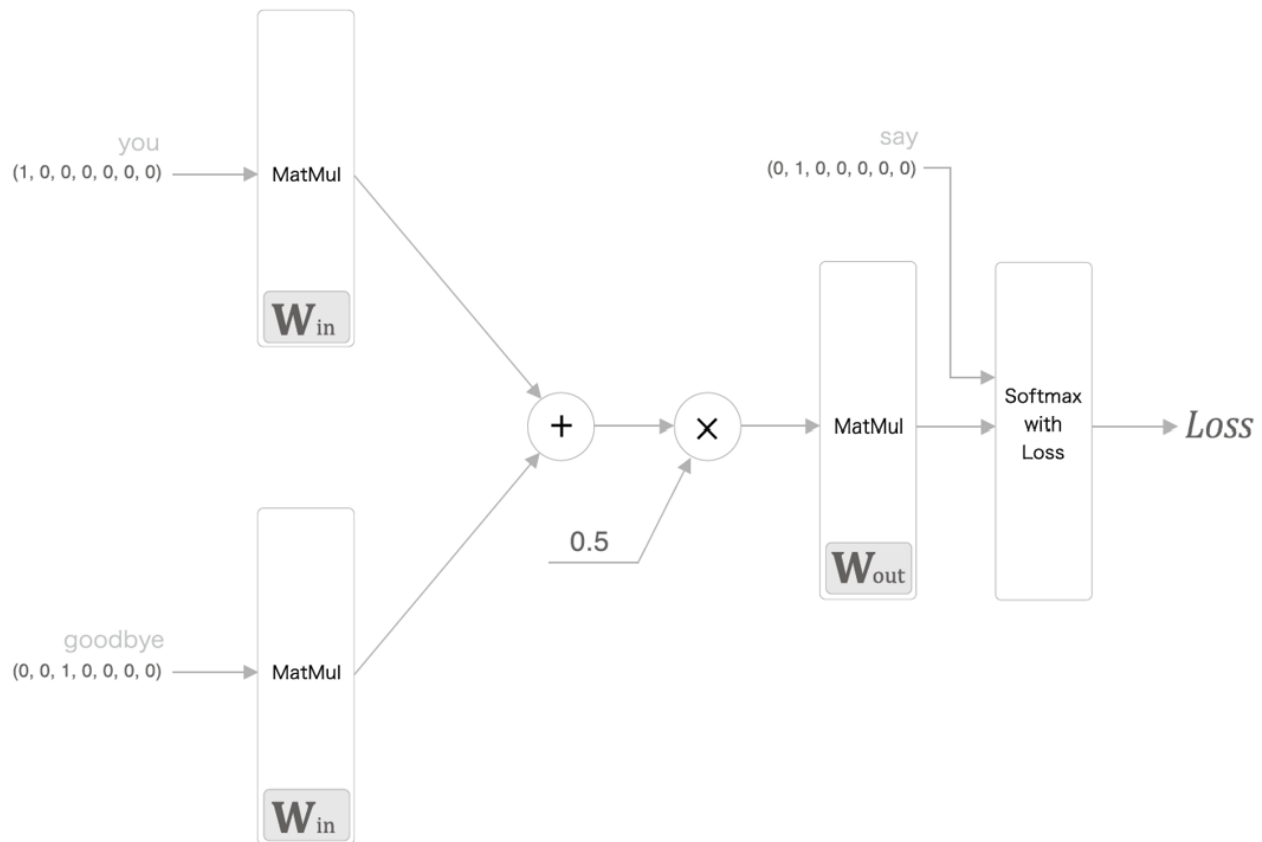
- 다중 클래스 분류를 수행하는 신경망
 - 신경망을 학습하려면 소프트맥스와 교차 엔트로피 오차만 이용
 - 소프트맥스 함수를 이용해 점수를 확률로 변환
 - 확률과 정답 레이블로부터 교차 엔트로피 오차를 구함
 - 오차 값을 손실로 사용해 학습을 진행

그림 3-13 CBOW 모델의 학습 시 신경망 구성



3.2.2 CBOW 모델의 학습(2)

- CBOW 모델 학습 시 신경망
 - Softmax 계층과 Cross Entropy Error 계층을 Softmax with Loss 계층으로 합침
 - 신경망의 순전파 - 손실을 얻을 수 있음



3.2.3 word2vec의 가중치와 분산 표현

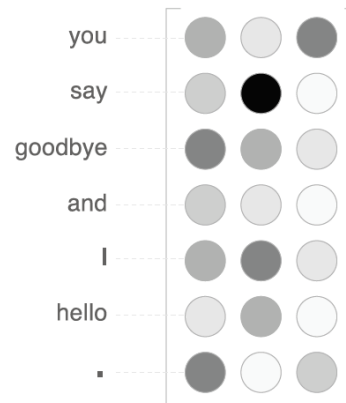
- word2vec 에 사용하는 신경망의 가중치
 - 입력 측 완전연결계층의 가중치(W_{in})와 출력 측 완전연결계층의 가중치(W_{out})
 - W_{in} 의 각 행이 각 단어의 분산 표현에 해당
 - W_{out} 는 단어의 의미가 인코딩된 벡터가 저장되고 있음 (분산 표현이 열 방향)

- 단어의 분산 표현으로 어떤 가중치를 선택할까?

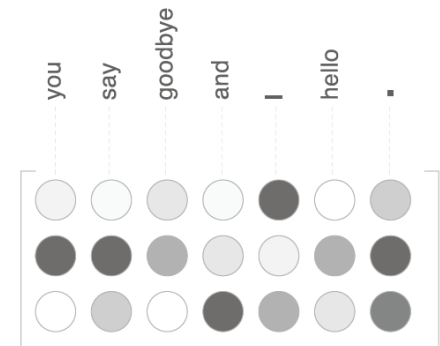
- 입력 측의 가중치만 이용
- 출력 측의 가중치만 이용
- 양쪽 가중치를 모두 이용

➔ word2vec 에서는
입력 측 가중치만 이용

그림 3-15 각 단어의 분산 표현은 입력 측과 출력 측 모두의 가중치에서 확인할 수 있다.



W_{in}
(7x3)



W_{out}
(3x7)

3.3 학습 데이터 준비

- word2vec 학습에 쓰일 학습 데이터 준비
- 한 문장짜리 말뭉치 이용
 - "You say goodbye and I say hello"

3.3.1 맥락과 타깃

- word2vec 에서 학습
 - 맥락 - word2vec 에서 이용하는 신경망의 입력
 - 타깃 - 정답 레이블은 맥락에 둘러싸인 중앙의 단어
 - 신경망에 맥락을 입력했을 때 타깃이 출현할 확률을 높이는 것임
- 말뭉치에서 맥락과 타깃을 만드는 작업

■ 말뭉치로부터 목표로 하는 단어를 타깃

- 주변 단어를 맥락
- 이 작업을 말뭉치 안의 모든 단어에 대해 수행
- 맥락의 각 행이 신경망의 입력
- 타깃의 각 행이 정답 레이블
- 맥락의 수는 여러 개가 될 수 있으나 타깃은 오직 하나뿐임

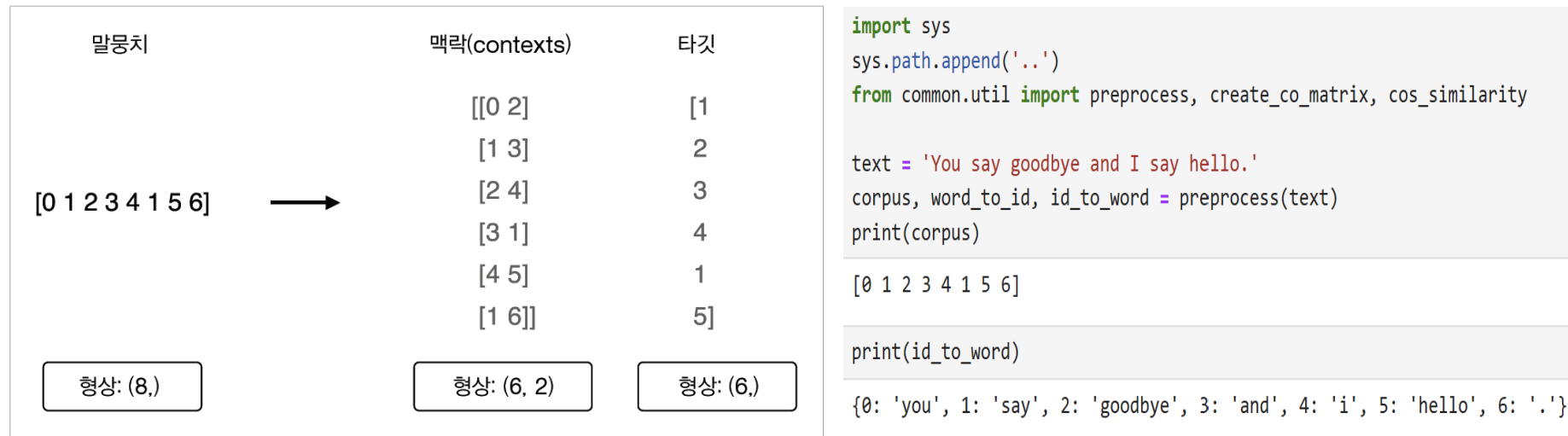
그림 3-16 말뭉치에서 맥락과 타깃을 만드는 예

말뭉치	맥락(contexts)	타깃
you <u>say</u> goodbye and I say hello .	you, goodbye	say
you say <u>goodbye</u> and I say hello .	say, and	goodbye
you say goodbye <u>and</u> I say hello .	goodbye, I	and
you say goodbye and <u>I</u> say hello .	and, say	I
you say goodbye and I <u>say</u> hello .	I, hello	say
you say goodbye and I say <u>hello</u> .	say, .	hello

3.3.1 맥락과 타깃(1)

- 말뭉치로부터 맥락과 타깃을 만드는 함수
 - 말뭉치 텍스트를 단어 ID로 변환 – preprocess() 함수 사용
 - 단어 ID의 배열인 corpus로부터 맥락과 타깃을 만들어 냄

그림 3-17 단어 ID의 배열인 corpus로부터 맥락과 타깃을 작성하는 예(맥락의 윈도우 크기는 1)



- 맥락은 2차원 배열
- context[0] 에는 0번째 맥락이 저장되고, context[1]에는 1번째 맥락이 저장
- 타깃에서도 target[0] 에는 0번째 타깃이, target[1]에는 1번째 타깃이 저장

3.3.1 맥락과 타깃(2)

- 맥락과 타깃을 만드는 함수 구현
 - 함수 인수 – 단어 ID의 배열, 맥락의 윈도우 크기
 - 맥락과 타깃을 각각 넘파이 다차원 배열로 돌려줌
- 말뭉치로부터 맥락과 타깃을 만들어내는 코드
 - 맥락과 타깃의 각 원소는 단어 ID

```
def create_contexts_target(corpus, window_size=1):  
    target = corpus[window_size:-window_size]  
    contexts = []  
  
    for idx in range(window_size, len(corpus)-window_size):  
        cs = []  
        for t in range(-window_size, window_size + 1):  
            if t == 0:  
                continue  
            cs.append(corpus[idx + t])  
        contexts.append(cs)  
  
    return np.array(contexts), np.array(target)
```

```
context, target = create_contexts_target(corpus, window_size=1)  
  
print(context)  
[[0 2]  
 [1 3]  
 [2 4]  
 [3 1]  
 [4 5]  
 [1 6]]  
  
print(target)  
[1 2 3 4 1 5]
```

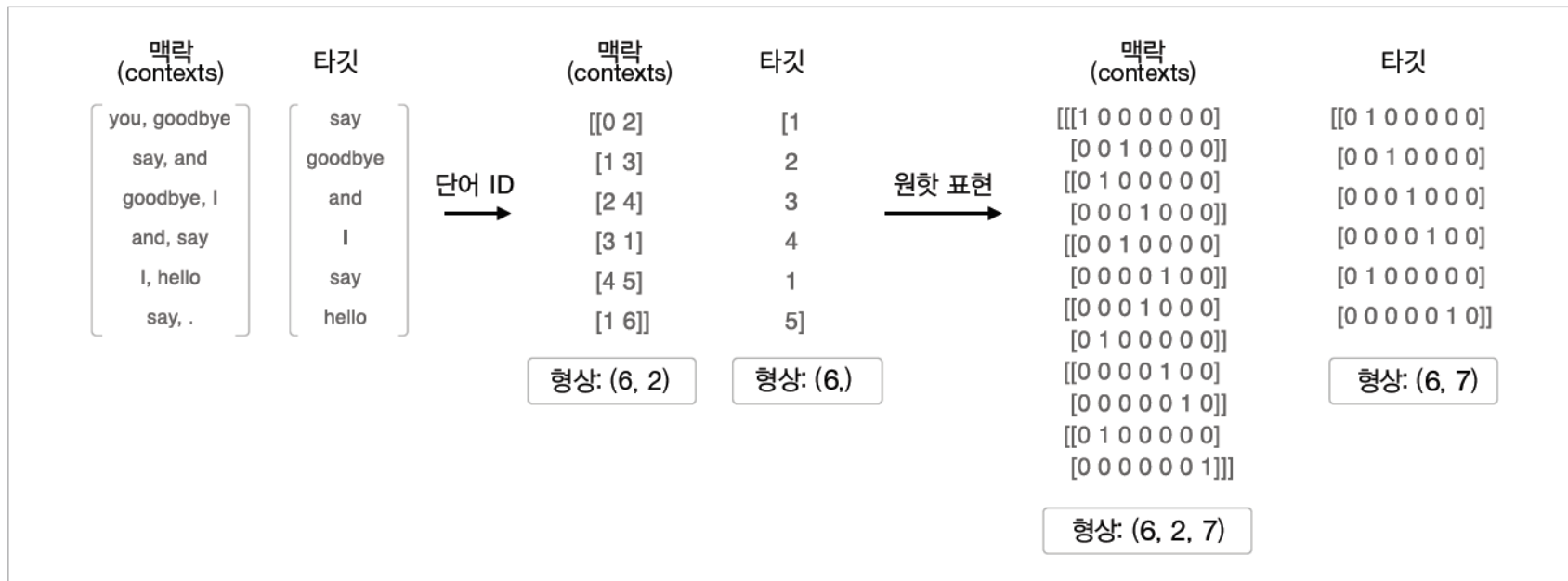

3.3.2 원핫 표현으로 변환

■ 맥락과 타깃을 원핫 표현으로 변환

- 맥락과 타깃을 단어 ID에서 원핫 표현으로 변환
- 각각의 다차원 배열의 형상에 주목해야 함

(단어 ID 이용시 맥락 형상은 (6, 2) 인데 원핫 표현으로 변환하면 (6, 2, 7) 이 됨)

그림 3-18 '맥락'과 '타깃'을 원핫 표현으로 변환하는 예



3.3.2 원핫 표현으로 변환(1)

■ 함수 구현

- 인수로 단어 ID 목록과 어휘 수를 받음

```
import sys
sys.path.append('.')
from common.util import preprocess, create_context_target, convert_one_hot

text = 'You say goodbye and I say hello.'
corpus, word_to_id, id_to_word = preprocess(text)

context, target = create_contexts_target(corpus, window_size=1)

vocab_size = len(word_to_id)
target = convert_one_hot(target, vocab_size)
contexts = convert_one_hot(contexts, vocab_size)
```

```
print(target)
```

```
[[0 1 0 0 0 0 0]
 [0 0 1 0 0 0 0]
 [0 0 0 1 0 0 0]
 [0 0 0 0 1 0 0]
 [0 1 0 0 0 0 0]
 [0 0 0 0 0 1 0]]
```

```
print(contexts)
```

```
[[[1 0 0 0 0 0 0]
   [0 0 1 0 0 0 0]]

 [[0 1 0 0 0 0 0]
   [0 0 0 1 0 0 0]]

 [[0 0 1 0 0 0 0]
   [0 0 0 0 1 0 0]]

 [[0 0 0 1 0 0 0]
   [0 1 0 0 0 0 0]]

 [[0 0 0 0 1 0 0]
   [0 0 0 0 0 1 0]]

 [[0 1 0 0 0 0 0]
   [0 0 0 0 0 0 1]]]
```

3.4 CBOW 모델 구현

- 신경망을 SimpleCBOW 구현
 - 넘파이 배열의 타입을 `astype('f')`로 지정 - 32비트 부동소수점
 - 입력층의 맥락을 처리하는 MatMul 계층의 맥락에서 사용하는 단어의 수 만큼 만들어야 함
 - 입력 측 MatMul 계층들은 모두 같은 가중치를 이용하도록 초기화

```
import sys
sys.path.append('.')
import numpy as np
from common.layers import MatMul, SoftmaxWithLoss

class SimpleCBOW:
    def __init__(self, vocab_size, hidden_size):
        V, H = vocab_size, hidden_size

        # 가중치 초기화
        W_in = 0.01 * np.random.randn(V, H).astype('f')
        W_out = 0.01 * np.random.randn(H, V).astype('f')

        # 계층 생성
        self.in_layer0 = MatMul(W_in)
        self.in_layer1 = MatMul(W_in)
        self.out_layer = MatMul(W_out)
        self.loss_layer = SoftmaxWithLoss()

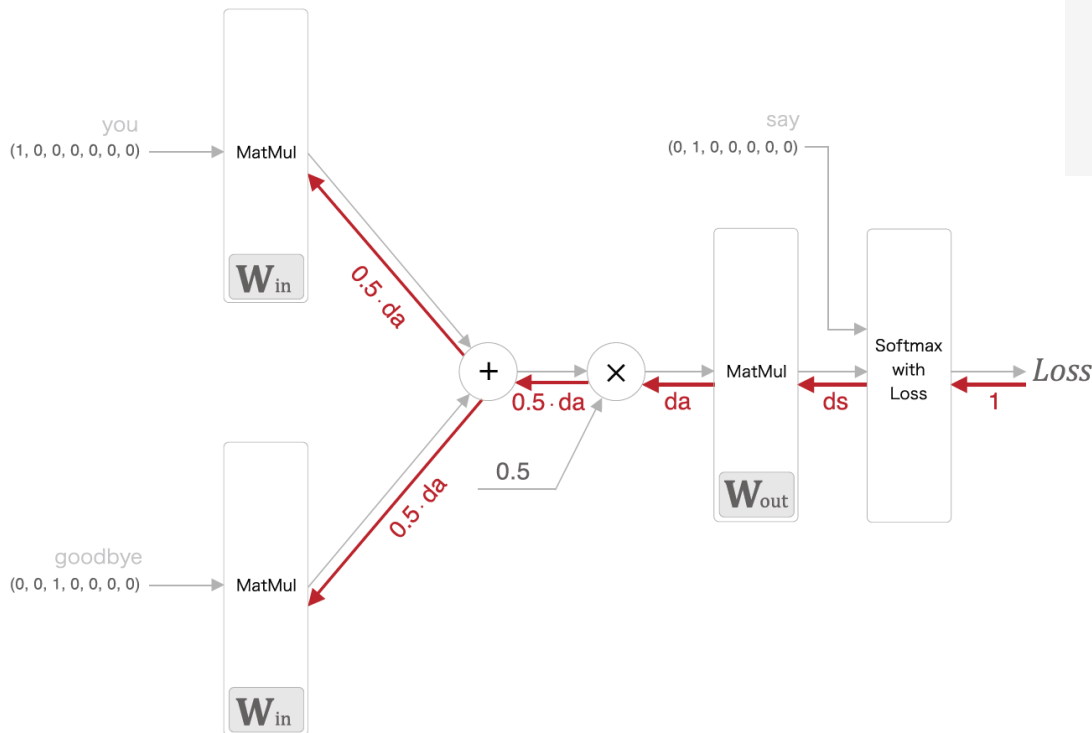
        # 모든 가중치와 기울기를 리스트에 모은다.
        layers = [self.in_layer0, self.in_layer1, self.out_layer]
        self.params, self.grads = [], []
        for layer in layers:
            self.params += layer.params
            self.grads += layer.grads

        # 인스턴스 변수에 단어의 분산 표현을 저장한다.
        self.word_vecs = W_in
```

3.4 CBOW 모델 구현 (1)

- 신경망 순전파 forward() 구현
 - 인수로 맥락과 타깃을 받아 손실을 반환
 - 인수 context는 3차원 넘파이 배열
- 신경망 역전파 backward() 구현

그림 3-20 CBOW 모델의 역전파(역전파의 흐름은 두꺼운(붉은) 화살표로 표시)



```
def forward(self, contexts, target):
    h0 = self.in_layer0.forward(contexts[:, 0])
    h1 = self.in_layer1.forward(contexts[:, 1])
    h = (h0 + h1) * 0.5
    score = self.out_layer.forward(h)
    loss = self.loss_layer.forward(score, target)
    return loss
```

```
def backward(self, dout=1):
    ds = self.loss_layer.backward(dout)
    da = self.out_layer.backward(ds)
    da *= 0.5
    self.in_layer1.backward(da)
    self.in_layer0.backward(da)
    return None
```

3.4.1 학습 코드 구현

■ CBOW 모델의 학습

- 학습 데이터를 준비해 신경망에 입력
- 기울기를 구하고 가중치 매개변수를 순서대로 갱신함
- 매개변수 갱신 방법으로 Adam 사용

■ Trainer 클래스의 신경망 학습 방식

- 학습 데이터로부터 미니배치를 선택한

다음 신경망에 입력해 기울기를 구함

- 기울기를 Optimizer 에 넘겨

매개변수를 갱신 수행

```
import sys
sys.path.append('.') # 부모 디렉터리의 파일을 가져올 수 있도록 설정
from common.trainer import Trainer
from common.optimizer import Adam
from simple_cbow import SimpleCBOW
from common.util import preprocess, create_contexts_target, convert_one_hot

window_size = 1
hidden_size = 5
batch_size = 3
max_epoch = 1000

text = 'You say goodbye and I say hello.'
corpus, word_to_id, id_to_word = preprocess(text)

vocab_size = len(word_to_id)
contexts, target = create_contexts_target(corpus, window_size)
target = convert_one_hot(target, vocab_size)
contexts = convert_one_hot(contexts, vocab_size)

model = SimpleCBOW(vocab_size, hidden_size)
optimizer = Adam()
trainer = Trainer(model, optimizer)

trainer.fit(contexts, target, max_epoch, batch_size)
trainer.plot()
```

3.4.1 학습 코드 구현 (1)

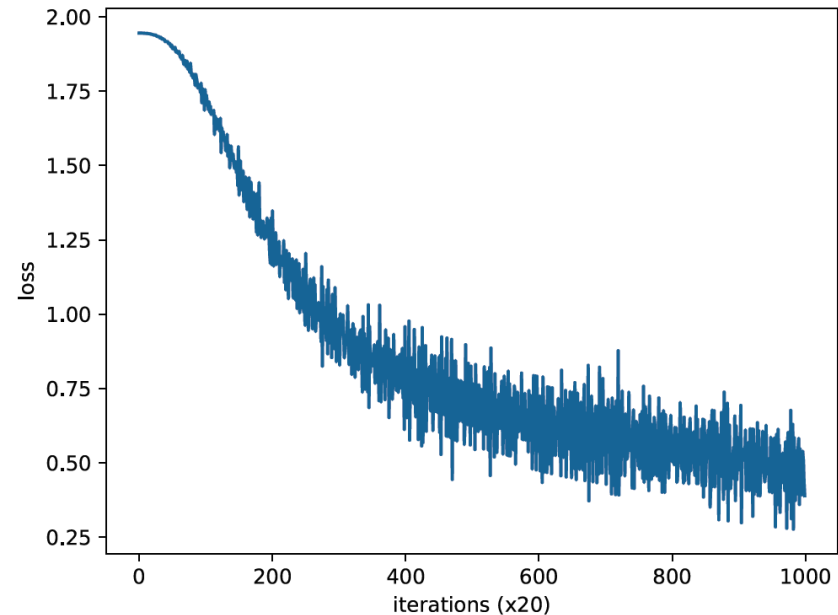
- 코드 실행 결과
 - 학습을 거듭할 수록 손실이 줄어듦
- 학습이 끝난 후의 가중치 매개변수
 - word_vecs의 각 행에는 대응하는 단어 ID의 분산 표현이 저장돼 있음

```
word_vecs = model.word_vecs
for word_id, word in id_to_word.items():
    print(word, word_vecs[word_id])
```

```
you [ 0.88823366  1.7157329  0.89106625 -0.896193  0.9120244 ]
say [-1.1830025  1.3346509 -0.90001035  1.1342818 -1.1407871 ]
goodbye [ 1.034126 -0.404376  1.0561082 -1.0722771  1.0040786 ]
and [-0.5968976  1.0529478 -1.8870403  0.7166605 -0.75690883]
i [ 1.0331293 -0.42127857  1.0579871 -1.0447063  1.0120072 ]
hello [ 0.9108599  1.7142811  0.8814803 -0.8984028  0.9181553 ]
. [-1.4094278  1.2020818  1.01185  1.2553884 -1.2697389]
```

- 단어를 밀집 벡터로 나타냄
 - 밀집 벡터가 단어의 분산 표현임
 - 분산 표현은 단어의 의미를 잘 파악한 벡터 표현임

그림 3-21 학습 경과를 그래프로 표시(가로축은 학습 횟수, 세로축은 손실)



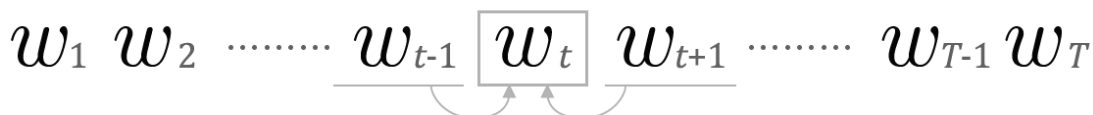
3.5 word2vec 보충

3.5.1 CBOW 모델과 확률

■ CBOW 모델 확률

- CBOW 모델이 하는 일은 맥락을 주면 타깃 단어가 출현할 확률

그림 3-22 word2vec의 CBOW 모델(맥락의 단어로부터 타깃 단어를 추측)



- 맥락으로 w_{t-1} 과 w_{t+1} 이 주어졌을 때 타깃이 w_t 가 될 확률

$$P(w_t | w_{t-1}, w_{t+1})$$

- 교차 엔트로피 오차를 적용

(문제의 정답은 w_t 가 발생이므로 w_t 에 해당하는 원소만 1이고 나머지는 0)

$$L = -\log P(w_t | w_{t-1}, w_{t+1})$$

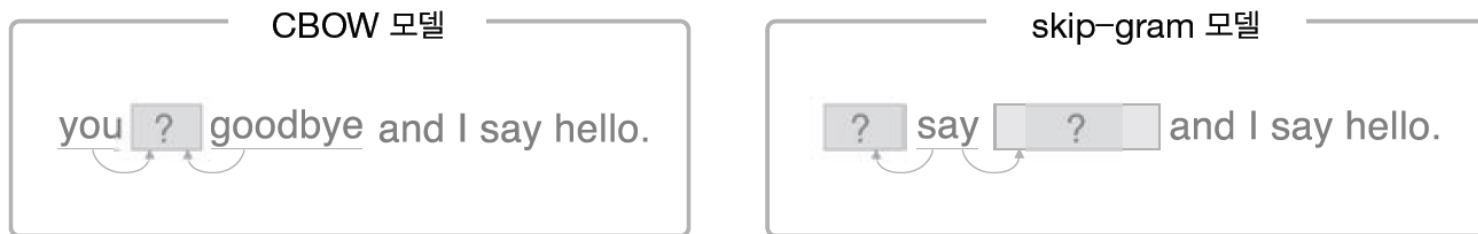
- 말뭉치 전체로 확장

$$L = -\frac{1}{T} \sum_{t=1}^T \log P(w_t | w_{t-1}, w_{t+1})$$

3.5.2 skip-gram 모델

- word2vec 에서 제안하는 2개 모델
 - CBOW 모델
 - skip-gram 모델 – CBOW 에서 다루는 맥락과 타깃을 역전시킨 모델

그림 3-23 CBOW 모델과 skip-gram 모델이 다루는 문제

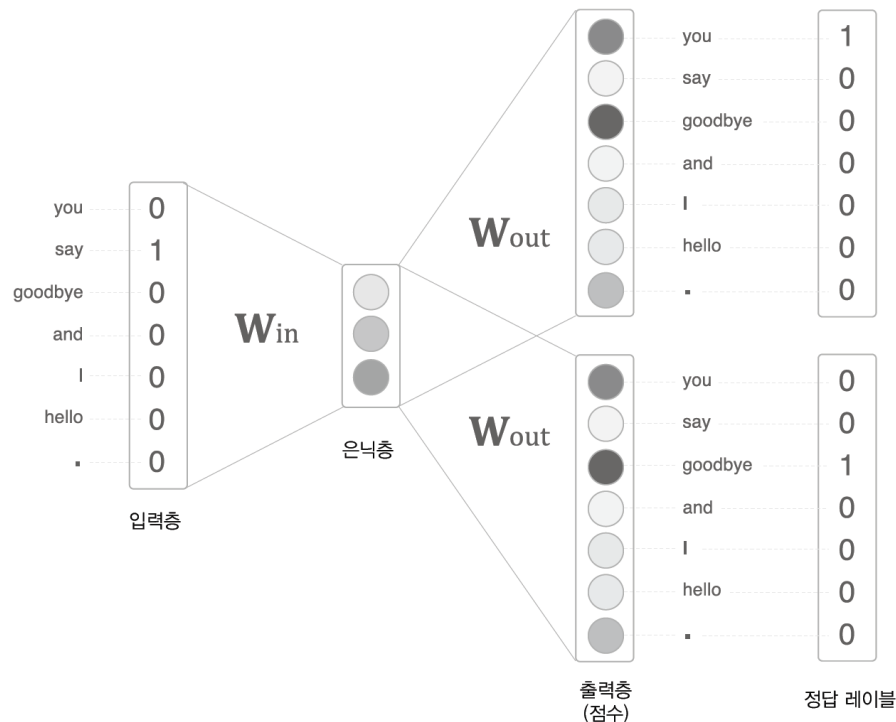


- CBOW 모델은 맥락이 여러 개 있고, 그 여러 맥락으로부터 중앙의 단어를 추측
- skip-gram 모델은 중앙의 단어(타깃)로부터 주변의 여러 단어(맥락)을 추측함

3.5.2 skip-gram 모델 (1)

- skip-gram 모델의 신경망 구성
 - 입력층은 하나
 - 출력층은 맥락의 수만큼 존재
 - 출력층에서는 (Softmax with Loss 계층) 개별적으로 손실을 구하고, 이 개별 손실들을 모두 더한 값을 최종 손실로 함

그림 3-24 skip-gram 모델의 신경망 구성 예



3.5.2 skip-gram 모델 (2)

- skip-gram 모델 확률 표기

$$P(w_{t-1}, w_{t+1} | w_t)$$

- w_t 가 주어졌을 때 w_{t-1} 과 w_{t+1} 이 동시에 일어날 확률
- skip-gram 모델에서는 맥락의 단어들 사이에 관련성이 없다고 가정하고 분해

$$P(w_{t-1}, w_{t+1} | w_t) = P(w_{t-1} | w_t) P(w_{t+1} | w_t)$$

- 교차 엔트로피 오차에 적용하여 skip-gram 모델의 손실 함수를 유도

$$\begin{aligned} L &= -\log P(w_{t-1}, w_{t+1} | w_t) \\ &= -\log P(w_{t-1} | w_t) P(w_{t+1} | w_t) \\ &= -(\log P(w_{t-1} | w_t) + \log P(w_{t+1} | w_t)) \end{aligned}$$

- skip-gram 모델의 손실 함수는 맥락별 손실을 구한 다음 모두 더함
- 샘플 데이터 하나짜리 skip-gram 의 손실 함수임

3.5.2 skip-gram 모델 (3)

- 말뭉치 전체로 확장한 skip-gram 모델의 손실 함수

$$L = -\frac{1}{T} \sum_{t=1}^T (\log P(w_{t-1} | w_t) + \log P(w_{t+1} | w_t))$$

- 맥락의 수만큼 추측하기 때문에 그 손실 함수는 각 맥락에서 구한 손실의 총합
- CBOW 모델은 타깃 하나의 손실을 구함
- CBOW 모델 vs skip-gram 모델
 - 단어 분산 표현의 정밀도 면에서 skip-gram 모델의 결과가 더 좋은 경우가 많음
 - 말뭉치가 커질수록 저빈도 단어나 유추 문제의 성능 면에서 skip-gram 모델이 더 뛰어난 경향이 있음
 - 학습 속도 면에서는 CBOW 모델이 더 빠름
 - skip-gram 모델은 손실을 맥락의 수만큼 구해야 해서 계산 비용이 커짐

3.5.4 통계 기반 vs 추론 기반

- 학습 측면
 - 통계 기반 기법은 말뭉치의 전체 통계에서 1회 학습하여 단어의 분산 표현을 얻음
 - 추론 기반 기법은 말뭉치를 일부를 사용하여 순차적 학습(미니배치 학습)
- 어휘에 추가할 새 단어가 생겨서 단어의 분산 표현을 갱신해야 하는 상황
 - 통계 기반 기법은 계산을 처음부터 다시 해야함
(동시발생 행렬을 다시 만들고 SVD를 수행하는 일련의 작업을 다시 수행)
 - 추론 기반 기법은 학습된 가중치를 초깃값으로 사용해 다시 학습함
(기존에 학습한 경험을 해치지 않으면서 단어의 분산 표현을 효율적으로 갱신)
- 단어의 분산 표현의 성격이나 정밀도
 - 통계 기반 기법은 주로 단어의 유사성이 인코딩됨
 - 추론 기반 기법은 단어의 유사성은 물론 단어 사이의 패턴까지도 인코딩됨
 - 정확성은 추론 기반과 통계 기반 기법의 우열을 가릴 수 없음
- 통계 기반 기법과 추론 기반 기법은 서로 관련되어 있음
 - 추론 기반 기법 모델도 말뭉치 전체의 동시발생 행렬에 특수한 행렬 분해를 적용한 것과 같음



Thank You

